

EAST AFRICA UNIVERSITY – GAROWE
FACULTY OF TECHNOLOGY AND ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE

REAL ESTATE
PROJECT MANAGEMENT
SYSTEM

A Thesis Submitted to the Faculty of Technology and Engineering in Partial
Fulfilment of the Requirements for the Degree of Bachelor of Science in
Computer Science

Prepared by:

Maxamuud Abdiqaadir Maxamed

Supervised by:

Mr. Cabdirisaaq Maxamed Cabdi Dharaar (Jirde)

Garowe, Somalia
September 2026

BONAFIDE CERTIFICATE

This is to certify that the thesis titled “**Real Estate Project Management System**” is a bona fide record of the project work carried out by **Maxamuud Abdiqaadir Maxamed** under my supervision, in partial fulfilment of the requirements for the award of the Degree of Bachelor of Science in Computer Science at East Africa University – Garowe, Faculty of Technology and Engineering.

The work presented in this thesis has not been submitted, in part or in full, for the award of any other degree or diploma of this or any other institution.

Maxamuud Abdiqaadir Maxamed
Student

Mr. Cabdirisq Maxamed Cabdi
Dharaar
Supervisor

DECLARATION

I, Maxamuud Abdiqaadir Maxamed, declare that this thesis, titled “Real Estate Project Management System,” is my own original work and has not been presented for the award of any degree or professional certificate in any other university or institution of higher learning. Where other sources have been consulted, they have been duly acknowledged in the text and listed in the reference section.

Signature:

Date:

APPROVAL PAGE

This thesis has been submitted for examination with my approval as the university supervisor.

Mr. Cabdirisag Maxamed Cabdi Dharaar (Jirde)
Supervisor

Signature:

Date:

Head of Department, Computer Science

Signature:

Date:

Dean, Faculty of Technology and Engineering

Signature:

Date:

ACKNOWLEDGEMENT

All praise is due to Allah, the Almighty, for granting me the health, patience, and ability to complete this thesis successfully.

I would like to extend my sincere gratitude to my supervisor, Mr. Cabdirisag Maxamed Cabdi Dharaar (Jirde), for his continuous guidance, constructive criticism, and encouragement throughout the course of this research and system development. His insights were of great value at every stage of the project.

I am also grateful to the entire teaching staff of the Department of Computer Science, Faculty of Technology and Engineering, East Africa University – Garowe, for the knowledge and skills imparted to me throughout my period of study.

My deepest appreciation goes to my family and friends for their unwavering moral and financial support, and for their patience during the many hours dedicated to this work. Finally, I thank my classmates and everyone who, directly or indirectly, contributed to the successful completion of this thesis.

ABSTRACT

Real estate companies in developing regions, including Somalia, still rely heavily on manual, paper-based, and spreadsheet-driven methods to manage construction projects, property inventories, sales, rentals, and payment records. These manual approaches are slow, prone to human error, difficult to audit, and offer management no real-time visibility into business performance. This study set out to design and implement a web-based Real Estate Project Management System capable of digitising and centralising these core operations for a small to medium-sized real estate company. The specific objectives were to analyse the shortcomings of the existing manual system, design a relational database and role-based system architecture capable of supporting multiple categories of staff, implement a functional web application covering project, property, customer, sales, rental, payment, document, and notification management, and evaluate the resulting system through structured testing. An iterative, Agile-inspired methodology was adopted, in which requirements were gathered through informal interviews and observation of real estate office practices, followed by successive design, implementation, and testing cycles. The system was built using ASP.NET Core 9 MVC for the presentation and application layers, Entity Framework Core 9 with a Code-First approach for data access, and Microsoft SQL Server as the relational database engine, with ASP.NET Core Identity providing authentication and role-based authorisation for five distinct user roles. The completed system automatically updates property availability when a sale or rental is recorded, recalculates outstanding balances whenever a payment is registered or removed, and raises notifications for pending payments, contracts nearing expiry, and delayed projects. Unit-level verification of the business rules and user-acceptance testing carried out with the demonstration accounts confirmed that all core workflows performed as designed, indicating a marked improvement in accuracy, speed, and traceability.

compared with the manual process it replaces. The study concludes that a moderately-scoped, role-aware web application can materially improve the operational efficiency of a small real estate business, and it recommends further work on mobile access, SMS and e-mail notification channels, and multi-branch support.

TABLE OF CONTENTS

BONAFIDE	CERTIFICATE	
.....	i	
DECLARATION		
.....	ii	
APPROVAL	PAGE	
.....	iii	
ACKNOWLEDGEMENT		
.....	iv	
ABSTRACT		
.....	v	
LIST	OF	TABLES
.....		vii
LIST	OF	FIGURES
.....		viii
LIST	OF	ABBREVIATIONS
.....		x
CHAPTER	ONE:	INTRODUCTION
.....		1
1.1	Background	of the study
.....		1
1.2	Problem	statement
.....		2

1.3	Objectives	of	the	study	3
1.4	Scope	and	limitations	of	the study
1.5	Significance	of	the	study	4
1.6	Organisation	of	the	thesis	5
CHAPTER TWO: LITERATURE REVIEW					6
2.1	Review	of	related	studies	6
2.2	Existing	systems	and	technologies	7
2.3	Research	gap			9
2.4	Conceptual	framework			9
CHAPTER THREE: METHODOLOGY					11
3.1	Research	methodology			11
3.2	Tools	and	technologies	used	12

3.3	Data	collection	methods	
				13
3.4	Data	analysis	techniques	
				14
3.5	System	design	approach	
				14
CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN				
				16
4.1	Requirements		specification	
				16
4.2	System		architecture	
				18
4.3	Use	case	diagram	
				19
4.4	Data	flow	and entity relationship diagram	
				20
4.5	Database		design	
				21
4.6	Implementation	of	modules	
				24
CHAPTER FIVE: SYSTEM IMPLEMENTATION AND TESTING				
				26
5.1	Introduction			
				26

5.2	Testing	
.....		
. 26		
5.3	Screenshots and explanation	
.....		28
CHAPTER SIX: CONCLUSION AND RECOMMENDATIONS		
.....		36
6.1	Conclusion	
.....		36
6.2	Summary of findings	
.....		36
6.3	Recommendations and suggestions	
.....		37
6.4	Future enhancement	
.....		37
REFERENCES		
.....		38
APPENDICES		
.....		39

(Note: right-click on this list in Microsoft Word and choose “Update Field > Update entire table” to regenerate accurate page numbers once the document is paginated.)

LIST OF TABLES

Table 3.1:	Hardware and software requirements	13
Table 4.2:	Attributes of the Property entity	21
Table 4.3:	Attributes of the Customer entity	22
Table 4.4:	Attributes of the Sale entity	22
Table 4.5:	Attributes of the Rental entity	23
Table 4.6:	Attributes of the Payment entity	23
Table 4.7:	Attributes of the BookingRequest entity	24
Table 4.8:	System user roles and permissions	24
Table 4.1:	Summary of functional requirements	17
Table 5.1:	Sample unit test cases and results	27
Table 5.2:	Sample user acceptance testing results	28

LIST OF FIGURES

Figure 3.1: Iterative Agile development cycle adopted for the system	15
Figure 4.1: Three-tier architecture of the system	18
Figure 4.2: Use case diagram of the system	19
Figure 4.3: Entity relationship diagram of the system	20
Figure 5.1: Login screen	28
Figure 5.2: Administrator dashboard	29
Figure 5.3: Properties list	29
Figure 5.4: Add new property form	30
Figure 5.5: Property details view	30
Figure 5.6: Customers list	31
Figure 5.7: Sales management	31

Figure	5.8:	Rentals	management
			 32
Figure	5.9:	Payments	list
			 32
Figure	5.10:	Projects	list
			 33
Figure	5.11:	Project	tasks
			 33
Figure	5.12:	Documents	list
			 34
Figure	5.13:	Reports	hub
			 34
Figure	5.14:	Users	management
			 35
Figure	5.15:	Notifications	panel
			 35

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
APA	American Psychological Association
CDP	Chrome DevTools Protocol
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
EF Core	Entity Framework Core
ERD	Entity Relationship Diagram
MVC	Model-View-Controller
PMS	Project Management System
SDLC	Software Development Life Cycle
SQL	Structured Query Language
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience

UML	Unified Modeling Language
-----	---------------------------

CHAPTER ONE: INTRODUCTION

1.1 Background of the study

The real estate sector is one of the fastest growing industries in Somalia, driven by rapid urbanisation, a large diaspora community investing in property, and the continuous expansion of residential and commercial developments in cities such as Garowe, Bosaso, and Mogadishu. Real estate companies in this environment are typically responsible for several interrelated activities at the same time: planning and monitoring construction projects, maintaining an inventory of individual properties within each project, recording sales and rental agreements with customers, tracking instalment and rent payments over long periods, and producing management reports that support pricing and investment decisions.

In many small and medium-sized real estate companies operating locally, these activities are still coordinated using a combination of paper files, register books, and general-purpose spreadsheet programs such as Microsoft Excel. Although spreadsheets are flexible and familiar to office staff, they were not designed to enforce the business rules that a real estate company depends on, for example, ensuring that a property cannot be sold twice, or that a customer's outstanding balance is always consistent with the payments that have actually been received. As a company grows and the number of active projects, properties, and customers increases, spreadsheet-based tracking becomes increasingly difficult to maintain, and the risk of duplicated records, lost files, and calculation errors increases correspondingly.

Advances in web application frameworks and relational database technology now make it realistic for a company of modest size to adopt a dedicated, purpose-built information system rather than continuing to adapt general-purpose office tools to

specialised business needs. Modern frameworks such as ASP.NET Core, combined with mature object-relational mapping tools such as Entity Framework Core and reliable database engines such as Microsoft SQL Server, allow a single developer or a small team to build a secure, role-aware, multi-user system within an academic project timeline. This thesis documents the design and implementation of such a system, referred to throughout as the Real Estate Project Management System, which was built as the practical component of this Bachelor of Science degree in Computer Science.

The system that resulted from this work centralises project, property, customer, sales, rental, payment, document, and reporting functions into a single web application, accessible to different categories of staff according to clearly defined roles. It was designed with a real, small real estate company's day-to-day operations in mind, drawing on informal observation of how such companies currently keep records, and on the functional gaps that this observation revealed.

1.2 Problem statement

Real estate companies that rely on manual or spreadsheet-based record keeping face a recurring set of operational problems. First, property status information is often out of date, because there is no single authoritative record that is automatically updated the moment a property is sold or rented; sales agents may therefore continue to market a property that a colleague has already sold, resulting in confusion and lost customer confidence. Second, payment tracking is largely manual, so outstanding balances on sales and rental contracts must be recalculated by hand whenever a new payment is received, which is time-consuming and susceptible to arithmetic mistakes. Third, there is no automatic mechanism to warn management about payments that are overdue, contracts that are close to expiry, or construction projects that are falling behind schedule, so such issues are frequently

discovered only after they have already caused financial or reputational harm. Fourth, important documents such as sales contracts, rental agreements, and payment receipts are stored in scattered physical folders or personal computers, making them difficult to retrieve quickly when needed for an audit, a customer dispute, or a management review.

Taken together, these problems mean that decision-makers in a growing real estate company cannot rely on their existing tools to answer, at any given moment, simple but important questions: how many properties remain available for sale, how much revenue has been collected against how much is still outstanding, which contracts require urgent attention, and how each ongoing project is progressing against its planned schedule and budget. The central problem addressed by this study is therefore the absence of an integrated, rule-enforcing information system that can answer these questions accurately and immediately, while remaining simple enough for non-technical staff to use on a daily basis.

1.3 Objectives of the study

1.3.1 General objective

The general objective of this study is to design, develop, and evaluate a web-based Real Estate Project Management System that digitises and integrates the core project, property, sales, rental, payment, and reporting operations of a small to medium-sized real estate company.

1.3.2 Specific objectives

The study was guided by the following specific objectives.

One, to review and document the shortcomings of the existing manual and spreadsheet-based methods used by real estate companies to manage projects, properties, and customer transactions.

Two, to design a relational database schema and a role-based system architecture capable of supporting the five categories of user identified during requirements analysis, namely Administrator, Project Manager, Sales Agent, Accountant, and Customer.

Three, to implement a functional web application that automates property status updates, sales and rental record keeping, payment balance recalculation, document storage, and system-generated notifications.

Four, to test the resulting system through unit-level verification of its core business rules and structured user acceptance testing, so as to confirm that it meets the objectives stated above.

1.4 Scope and limitations of the study

The system developed in this study covers project management, property inventory management, customer relationship records, sales transactions, rental contracts, payment processing and balance tracking, document storage and retrieval, dashboard reporting, and role-based user administration. It supports five user roles, namely Administrator, Project Manager, Sales Agent, Accountant, and Customer, each restricted to the functions relevant to that role through the authorisation features of ASP.NET Core Identity. The system was designed and tested as a single-organisation, single-branch application intended to run on a local or single-instance server deployment.

Several areas were deliberately left outside the scope of this study. The system does not include a public-facing property listing website for prospective buyers who are not yet registered customers, nor does it provide integration with external payment gateways, mobile money application programming interfaces, or short message service and electronic mail providers, although the architecture allows such integrations to be added in future work. Multi-branch or multi-company support, in which a single deployment would serve several independent real estate companies with isolated data, was likewise not implemented. Finally, because the system was developed and evaluated within an academic timeframe using demonstration data rather than a live production dataset, its performance under a very large number of concurrent users has not been verified and is identified as an area for future work in Chapter Six.

1.5 Significance of the study

This study is significant to several groups of stakeholders. For real estate company owners and managers, the system offers a practical, low-cost alternative to expensive commercial property management platforms, while still enforcing the core business rules that protect the company from double-selling properties or losing track of outstanding balances. For sales agents, accountants, and project managers, the system removes much of the manual arithmetic and cross-checking that their daily tasks currently require, allowing them to focus on customer service and project delivery instead of record keeping. For customers, the system creates the foundation for more transparent and timely communication regarding the status of their purchase or rental agreement.

From an academic perspective, this study demonstrates the practical application of software engineering principles, relational database design, and modern web development technology to a real, locally relevant business problem, in fulfilment

of the practical project requirement of the Bachelor of Science degree in Computer Science at East Africa University – Garowe. It also contributes a documented case study that future students in the Faculty of Technology and Engineering may reference when designing information systems for other sectors of the local economy.

1.6 Organisation of the thesis

This thesis is organised into six chapters. Chapter One has introduced the background, problem statement, objectives, scope, and significance of the study. Chapter Two reviews related literature and existing property management technologies, and identifies the research gap that this study addresses. Chapter Three describes the research methodology, the tools and technologies used, and the system design approach adopted during development. Chapter Four presents the system analysis and design, including the functional and non-functional requirements, system architecture, use case diagram, entity relationship diagram, and database design. Chapter Five documents the implementation and testing of the system, supported by annotated screenshots of the working application. Chapter Six concludes the thesis with a summary of findings, recommendations, and suggested directions for future enhancement.

CHAPTER TWO: LITERATURE REVIEW

2.1 Review of related studies

Software engineering literature has long emphasised that information systems perform best when they are built around a clear understanding of the business rules of the domain they serve, rather than being a direct digital copy of the paper forms they replace (Sommerville, 2016). This principle is particularly relevant to real estate management, where a small number of business rules, such as the fact that a single property cannot simultaneously be available, sold, and rented, must be enforced consistently across every part of the system rather than left to the discipline of individual staff members. Pressman and Maxim (2020) similarly argue that requirements engineering activities, including stakeholder observation and iterative validation, are essential before design work begins, because errors introduced at the requirements stage are the most expensive to correct once a system has been implemented.

Database-oriented literature reinforces this view from a data modelling perspective. Elmasri and Navathe (2016) describe how a well-normalised relational schema, together with foreign key constraints, can enforce referential integrity rules that would otherwise depend on manual discipline, for example guaranteeing that a payment record can never exist without a corresponding sale or rental agreement. This principle directly informed the database design decisions documented in Chapter Four of this thesis, where relationships between the Property, Sale, Rental, Payment, and Customer entities were deliberately constrained at the database level rather than relying solely on application-level checks.

On the architectural side, Fowler (2003) documents the Repository pattern as a means of separating an application's business logic from the details of how data is

persisted, which improves testability and allows the underlying data access technology to change with limited impact on the rest of the system. This pattern, together with a layered service architecture, is reflected in the architecture of the system developed in this study, described in detail in Section 4.2.

2.2 Existing systems and technologies

Several commercial property and real estate management platforms are available internationally, and reviewing their capabilities helps to position the contribution of this study. Yardi Voyager and AppFolio Property Manager are widely used cloud-based platforms that provide accounting, leasing, and maintenance-tracking functionality for large property portfolios, while Buildium targets small and medium-sized property managers with similar, though less extensive, functionality. These platforms are generally well engineered and feature-rich, but they are priced and licensed for markets with reliable, continuous internet connectivity and established banking integrations, and they are typically not tailored to the specific workflow of a real estate developer that both constructs projects and subsequently sells or rents the resulting units, as is common in the Somali market.

Beyond commercial software, the wider PropTech, meaning property technology, movement has been documented extensively; Baum (2020) traces how digital platforms have progressively been adopted across the property lifecycle, from marketing and transactions through to ongoing management, and observes that smaller markets and smaller companies frequently lag behind in adoption because available tools are not adapted to their scale or their local business practices. This observation is consistent with what was found during the requirements-gathering stage of this study, described in Chapter Three, where the real estate offices consulted continued to rely on spreadsheets specifically because available

commercial alternatives were considered too costly or too complex for their size of operation.

In the academic domain, numerous undergraduate and graduate information systems projects have addressed related problems, such as generic property listing websites or simple rental booking systems. However, such projects typically focus on a single transaction type, for example either sales or rentals, and rarely integrate project-level construction tracking, multi-role staff access, automatic payment recalculation, and system-generated notifications within a single coherent application, which is the combination of features that this study set out to deliver.

2.3 Research gap

The review above identifies a gap between, on one hand, feature-rich but costly commercial platforms that are not adapted to the scale or workflow of a small local developer-cum-agency, and, on the other hand, narrowly scoped academic prototypes that typically implement only one transaction type in isolation. Neither category adequately addresses the case of a company that both develops real estate projects and subsequently manages the resulting properties through sale or rental, tracks staged payments against each transaction, and requires role-based access for distinct categories of staff, all within a system that is inexpensive enough to deploy on a single local server. This study addresses that gap by combining project tracking, property inventory, sales, rentals, payments, document management, and role-based access control into a single, integrated web application built entirely on freely available, industry-standard technology.

2.4 Conceptual framework

The conceptual framework underlying this study treats the real estate company as a set of interconnected processes rather than a collection of independent record books. A construction project produces one or more properties; each property may, at any given time, be available, reserved, under construction, sold, or rented; a sale or a rental agreement, once created, generates an expected payment schedule against which actual payments are recorded over time; and the accumulation of these payments determines the payment status of the associated agreement. Documents, such as contracts and receipts, are attached to the relevant project, property, sale, rental, or customer record, and the system continuously evaluates this data to raise notifications when a defined condition is met, for example an approaching contract expiry date or an overdue payment. This conceptual framework, illustrated in the entity relationship diagram presented in Figure 4.3, directly shaped both the database design and the business logic implemented in the system's service layer, discussed in Sections 4.5 and 4.6.

CHAPTER THREE: METHODOLOGY

3.1 Research methodology

This study followed a design-based research methodology, in which the primary output is a working artefact, the Real Estate Project Management System itself, developed and refined through repeated cycles of requirement analysis, design, implementation, and evaluation. Design-based research is appropriate for information systems projects of this kind because it allows requirements to be adjusted as understanding of the problem domain deepens, rather than fixing every requirement before any implementation work begins, which is rarely realistic for a system whose future users could only be observed informally rather than engaged as a formal client. An Agile-inspired, iterative development approach was therefore combined with the overall design-based research strategy, and is described further in Section 3.5.

3.2 Tools and technologies used

The system was implemented using the technology stack summarised in Table 3.1. The back end and front end were both built using ASP.NET Core 9, a cross-platform, open-source web framework maintained by Microsoft, using the Model-View-Controller, commonly abbreviated as MVC, architectural pattern. Data access was implemented using Entity Framework Core 9 with a Code-First approach, in which the database schema is generated automatically from strongly typed C# entity classes and evolved over time using migrations. Microsoft SQL Server was selected as the relational database management system because of its strong integration with the .NET ecosystem and its suitability for the Windows-based deployment environment available for this project. User interface pages were built with Razor views styled using Bootstrap 5 and Bootstrap Icons, and dashboard

charts were rendered using Chart.js. ASP.NET Core Identity supplied user registration, authentication, and role-based authorisation. Server-side PDF export was implemented using QuestPDF, and spreadsheet export was implemented using ClosedXML, both of which are widely used open-source .NET libraries.

Table 3.1: Hardware and software requirements

Category	Requirement
Operating system	Windows 10/11 or any operating system capable of hosting the .NET 9 runtime
Back-end framework	ASP.NET Core 9 MVC (C#)
Data access	Entity Framework Core 9 (Code-First, with migrations)
Database server	Microsoft SQL Server (LocalDB, Express, or full instance)
Front-end libraries	Bootstrap 5, Bootstrap Icons, Chart.js, jQuery
Authentication	ASP.NET Core Identity (cookie-based authentication, role-based authorisation)
Reporting / export	QuestPDF (PDF export), ClosedXML (Excel export)
Version control	Git repository with incremental commits during development
Development environment	Visual Studio / .NET SDK 9.0, with the project organised as a single ASP.NET Core MVC solution

Minimum hardware	Dual-core processor, 4 GB RAM, 2 GB free disk space (server); any modern browser (client)
------------------	---

3.3 Data collection methods

Requirements for the system were gathered mainly through informal, unstructured interviews with individuals familiar with real estate office operations, supplemented by direct observation of the spreadsheets and paper forms currently used to track projects, properties, and customer payments. This qualitative approach was chosen because no formal client organisation was available to complete detailed written requirement questionnaires within the academic timeframe of the project; instead, the researcher relied on domain knowledge built up through these conversations and observations, cross-checked against the functionality offered by the commercial platforms reviewed in Section 2.2. Document review of sample sales contracts, rental agreements, and payment receipts also informed the fields captured in the Sale, Rental, Payment, and Document entities described in Chapter Four.

3.4 Data analysis techniques

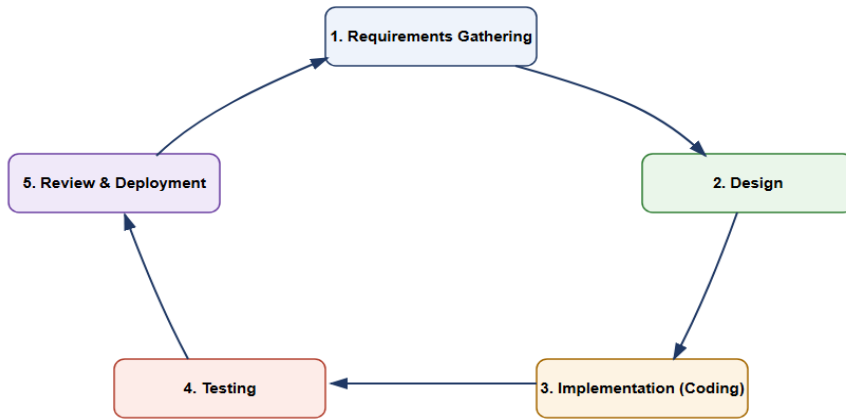
The information gathered during data collection was analysed qualitatively rather than statistically, given the exploratory nature of the requirements-gathering process. Recurring themes, such as the repeated difficulty of tracking outstanding balances by hand and the absence of any automatic warning for expiring contracts, were extracted from the interview notes and observations and translated directly into functional requirements, listed in Table 4.7. Where two candidate designs were being compared, for example choosing between storing a payment status as a stored value or as a value calculated on demand, a lightweight cost-benefit comparison

was used, weighing implementation complexity against data consistency; this particular example is resolved in Section 4.6, where the decision to recalculate payment status automatically after every payment transaction is explained.

3.5 System design approach

The system was designed and built using an iterative, Agile-inspired software development life cycle rather than a strict sequential Waterfall model, because the requirements were expected to, and did, evolve as early versions of each module were reviewed. As illustrated in Figure 3.1, each iteration proceeded through five stages: requirements gathering for the module in question, design of the relevant entities, controllers, and views, implementation of the corresponding code, testing of the resulting functionality against the business rules identified for that module, and a review step in which the outcome was compared against the objectives in Section 1.3 before the next module was started. This approach allowed early modules, such as Property and Customer management, to directly inform the design of later, more dependent modules, such as Sales, Rentals, and Payments, which build upon the two former entities. Stellman and Greene (2014) describe this style of incremental delivery as particularly well suited to small teams and single-developer projects, since it produces a usable, demonstrable increment of the system after every cycle rather than only at the very end of the project.

Iterative Agile Development Cycle Adopted for the System



Each iteration (sprint) refined one or more modules — Projects, Properties, Sales, Rentals, Payments, Reports — based on supervisor and user feedback.

Figure 3.1: Iterative Agile development cycle adopted for the system

CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN

4.1 Requirements specification

4.1.1 Functional requirements

The functional requirements of the system, summarised in Table 4.1, describe the specific tasks that the system must be able to perform for its different categories of user. These requirements were derived directly from the problems identified in Section 1.2 and were validated against the module-by-module review carried out at the end of each development iteration, as described in Section 3.5.

Table 4.1: Summary of functional requirements

ID	Requirement	Description
FR-01	User authentication	The system shall allow registered users to log in with an e-mail or username and password, and shall restrict access to functionality according to the user's assigned role.
FR-02	Project management	The system shall allow authorised staff to create, update, and monitor real estate projects, including code, location, budget, schedule, and progress percentage.
FR-03	Property management	The system shall allow authorised staff to register properties under a project, record property attributes, upload property images, and track property status.

FR-04	Customer management	The system shall allow authorised staff to register and maintain customer profiles, including contact and identification details.
FR-05	Sales processing	The system shall allow authorised staff to record a property sale, and shall automatically mark the corresponding property as sold and record any deposit as an initial payment.
FR-06	Rental processing	The system shall allow authorised staff to record a rental agreement, and shall automatically mark the corresponding property as rented and record any deposit as an initial payment.
FR-07	Payment tracking	The system shall allow authorised staff to record and delete payments against a sale or rental, and shall automatically recalculate the remaining balance and payment status after each change.
FR-08	Document management	The system shall allow authorised staff to upload and retrieve documents associated with a project, property, sale, rental, or customer.
FR-09	Reporting	The system shall generate sales, revenue, property, customer, payment, project-progress, and rental reports, exportable to PDF and Excel formats.
FR-10	Dashboard	The system shall present a role-appropriate dashboard summarising key indicators, including totals for projects, properties, sales, revenue, and pending payments.
FR-11	Notifications	The system shall automatically generate notifications for pending payments, contracts expiring within thirty days, and delayed projects.

FR-12	User and role management	The system shall allow an administrator to create user accounts and assign each account to one of the five defined roles.
FR-13	Booking requests	The system shall allow a registered customer to browse available properties and submit a purchase or rental booking request, and shall allow authorised staff to review, confirm, or reject each request, automatically creating the corresponding sale or rental on confirmation.
FR-14	Role permission configuration	The system shall allow an administrator to configure, through a permission matrix, which of the functional modules each non-administrator role may access.
FR-15	Project status audit trail	The system shall automatically record a history entry, together with the reason given and the user responsible, whenever a project's status or progress percentage is changed.
FR-16	Notification preferences	The system shall allow a user to enable or disable, for their own account, each individual category of system-generated notification.
FR-17	Public property catalogue	The system shall present an anonymous, public-facing catalogue of available properties to prospective customers who have not yet registered an account.

4.1.2 Non-functional requirements

In addition to the functional requirements above, the system was designed to satisfy several non-functional requirements. With respect to security, passwords are never stored in plain text, being instead hashed by ASP.NET Core Identity, and every controller action is protected by role-based authorisation so that, for example, only

staff in the Accountant or Administrator roles may record payments. With respect to usability, the interface was built using the Bootstrap 5 framework to ensure a consistent, responsive layout across desktop and tablet screen sizes. With respect to maintainability, business logic was placed in dedicated service classes, described in Section 4.6, rather than directly inside controllers, so that individual modules can be modified with limited risk of unintended side effects elsewhere in the system. With respect to reliability, critical operations such as recording a sale and its associated deposit payment are performed within the same logical unit of work, so that the property status and the payment record are always kept consistent with one another.

4.2 System architecture

The system follows a layered, three-tier architecture, illustrated in Figure 4.1. The presentation layer consists of Razor views rendered by the browser, styled with Bootstrap 5 and enhanced with Chart.js visualisations and a reusable notification-bell view component. The application, or business logic, layer consists of MVC controllers, one for each major module, together with dedicated service classes that encapsulate the business rules governing sales, rentals, payments, notifications, dashboard statistics, file storage, and report export; ASP.NET Core Identity also resides in this layer, governing authentication and authorisation. The data access layer consists of a generic repository abstraction, defined by the `IRepository<T>` interface and its `Repository<T>` implementation, together with Entity Framework Core's `ApplicationDbContext`, which mediates all communication with the underlying Microsoft SQL Server database. Uploaded files, such as property images and supporting documents, are stored on the local file system under the `wwwroot/uploads` directory and served as static files rather than being stored as binary data inside the database, which keeps the database itself compact and

improves the performance of file retrieval. Access to each functional module is additionally governed by a custom authorisation filter, `ModulePermissionAttribute`, which consults an administrator-configurable permission matrix before allowing a request to reach the corresponding controller action. Although a generic repository abstraction, `IRepository<T>`, is also defined for potential reuse, most controllers and services interact with `ApplicationDbContext` directly, favouring the additional query flexibility this affords.

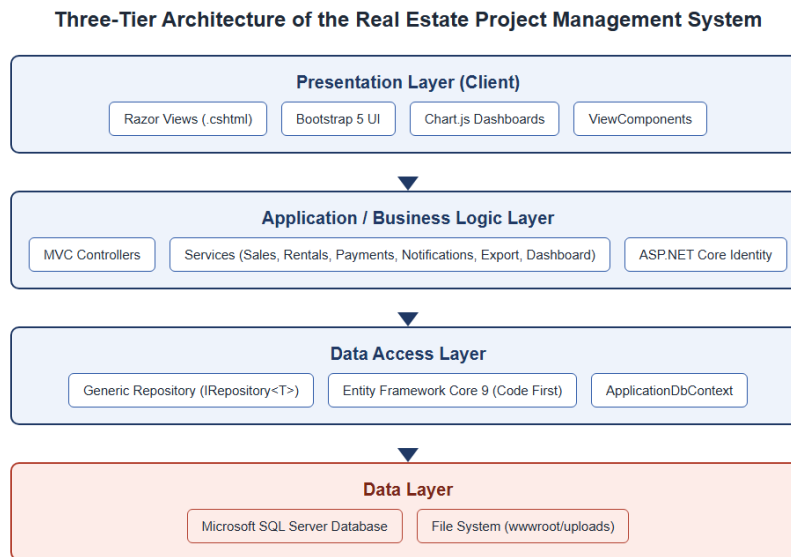


Figure 4.1: Three-tier architecture of the system

4.3 Use case diagram

Figure 4.2 presents the use case diagram of the system, showing the five actors identified during requirements analysis, namely Administrator, Project Manager, Sales Agent, Accountant, and Customer, together with the principal use cases each actor may perform. The Administrator has access to the complete set of use cases,

including user and role management, while the remaining staff roles are restricted to the use cases relevant to their responsibilities; for example, the Sales Agent may record a sale but may not manage user accounts, and the Accountant may record payments but may not create new properties. The Customer actor is limited to viewing the sale or rental agreement associated with that customer's own account. Every actor participates in the Login and Authenticate use case, since role-based access control is enforced at every controller action. In addition to these staff-oriented use cases, a Customer may browse a public catalogue of available properties and submit a booking request for a property of interest; a Project Manager or Sales Agent then reviews each pending request and either confirms it, which automatically creates the corresponding sale or rental record, or rejects it with a stated reason.

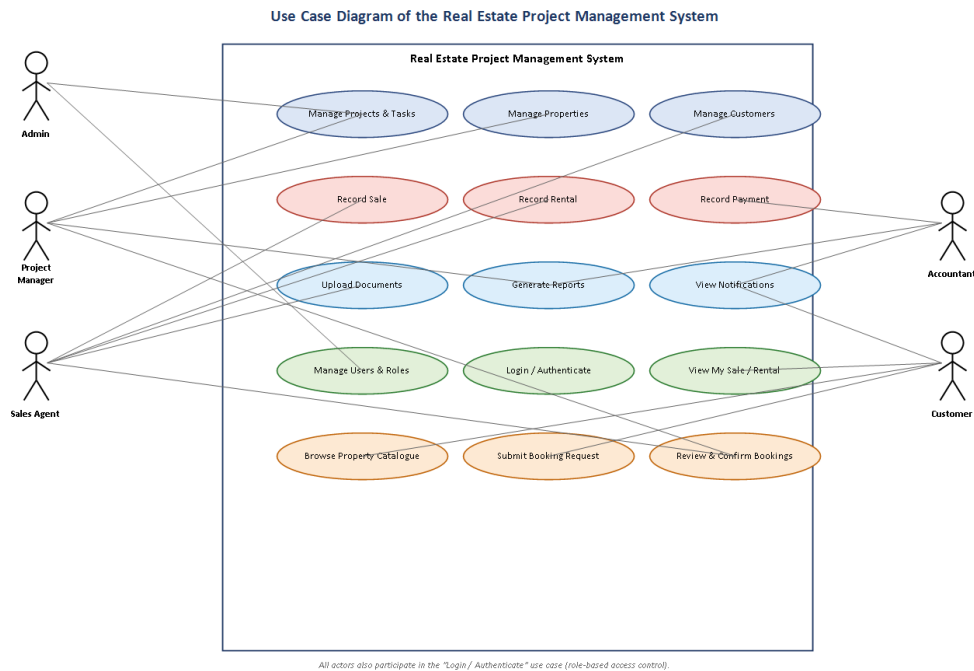


Figure 4.2: Use case diagram of the system

4.4 Data flow and entity relationship diagram

At a data flow level, a new property is created within the context of an existing project; when a sale or a rental is subsequently recorded against that property, the property's status is updated automatically and an initial payment is generated if a deposit was specified; every additional payment recorded against that sale or rental triggers a recalculation of the outstanding balance and the corresponding payment status. Figure 4.3 presents the simplified entity relationship diagram that models these flows at the database level. Each project may contain many properties and many project tasks, and is optionally supervised by one application user acting as project manager. Each property may be associated with, at most, one active sale and one active rental at any given time, reflecting the business rule that a single property cannot be sold and rented simultaneously. Each sale and each rental may accumulate many payment records over time, and each customer may be linked to many sales, rentals, and payments across the lifetime of the system. Documents and notifications are attached as supporting entities linked, respectively, to the relevant transactional record and to the application user who should receive the notification. A booking request, submitted by a Customer against a specific Property and optionally reviewed by an ApplicationUser, is linked, on confirmation, to at most one resulting Sale or Rental, reflecting the workflow through which a prospective transaction is proposed by a customer before being converted into a binding sale or rental agreement by staff.

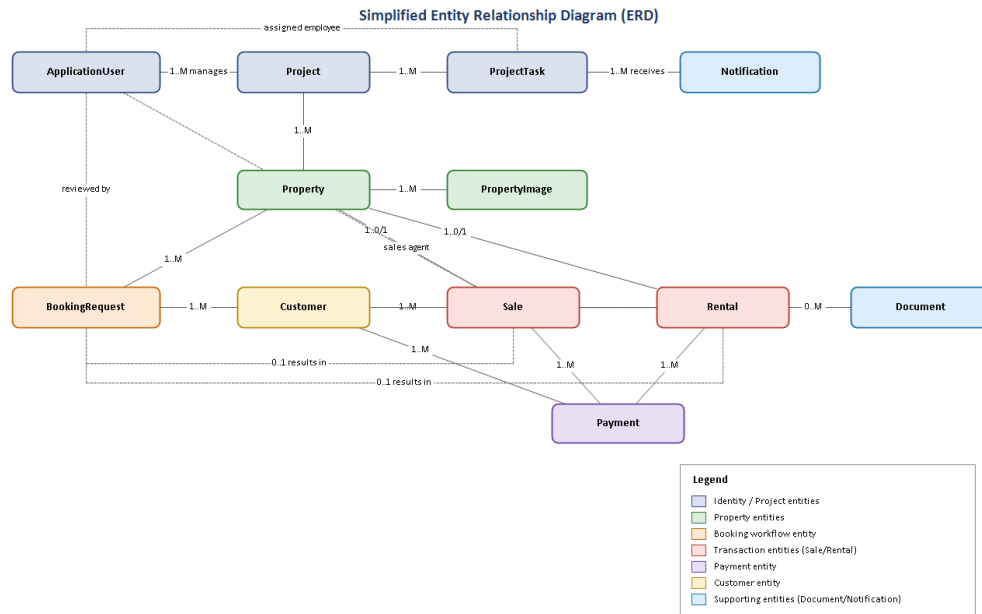


Figure 4.3: Entity relationship diagram of the system

4.5 Database design

The database was implemented using Entity Framework Core's Code-First approach, meaning that the schema shown in this section was generated from C# entity classes and evolved through migrations, rather than being hand-written in Structured Query Language. Tables 4.2 through 4.7 document the attributes of the six most central entities in the schema; the remaining entities, namely Project, ProjectTask, PropertyImage, Document, Notification, ApplicationUser, RolePermission, ProjectStatusHistory, and NotificationPreference, are described narratively in Section 4.6 alongside the module that principally uses them.

Table 4.2: Attributes of the Property entity

Attribute	Data type	Description
Id	int (PK)	Primary key, automatically generated
Code	string(30)	Unique property code, for example P-013
ProjectId	int (FK)	Foreign key referencing the owning Project
Type	enum	Apartment, Villa, Office, Shop, or Land
Building / Floor / UnitNumber	string	Physical location details within the project
Area	decimal(10,2)	Floor area of the unit, in square metres
Bedrooms / Bathrooms	int	Room counts, where applicable to the property type
Price	decimal(18,2)	Listed price of the property
Status	enum	Available, Reserved, Sold, Rented, or UnderConstruction
Description	string(2000)	Free-text description of the unit
CreatedDate	DateTime	Date the record was created

Table 4.3: Attributes of the Customer entity

Attribute	Data type	Description
-----------	-----------	-------------

Id	int (PK)	Primary key
Name	string(150)	Full name of the customer
Phone	string(30)	Contact telephone number
Email	string(150)	Optional e-mail address
Address / Nationality / IdNumber	string	Identification and contact details
DateOfBirth	DateTime	Optional date of birth
RegistrationDate	DateTime	Date the customer record was created
ApplicationUserId	string (FK)	Optional link to a self-service customer account

Table 4.4: Attributes of the Sale entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the buying Customer
PropertyId	int (FK)	Foreign key referencing the sold Property

SalesAgentId	string (FK)	Foreign key referencing the ApplicationUser who closed the sale
SaleDate	DateTime	Date the sale was recorded
TotalPrice / Discount / FinalPrice	decimal(18,2)	Agreed price, any discount applied, and the resulting final price
Deposit / RemainingBalance	decimal(18,2)	Initial deposit and the balance still outstanding
PaymentStatus	enum	Pending, Partial, Paid, or Overdue; recalculated automatically

Table 4.5: Attributes of the Rental entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the tenant Customer
PropertyId	int (FK)	Foreign key referencing the rented Property
MonthlyRent / Deposit	decimal(18,2)	Agreed monthly rent and deposit amount
StartDate / EndDate	DateTime	Contract period

PaymentStatus	enum	Pending, Partial, Paid, or Overdue
ContractStatus	enum	Active, Expired, Terminated, or Renewed

Table 4.6: Attributes of the Payment entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the paying Customer
SaleId / RentalId	int (FK, nullable)	Exactly one is populated, linking the payment to a sale or a rental
Amount	decimal(18,2)	Amount paid
PaymentDate	DateTime	Date payment was received
Method	enum	Cash, BankTransfer, MobileMoney, or Card
ReferenceNumber / Notes	string	Optional transaction reference and remarks

Table 4.7: Attributes of the BookingRequest entity

Attribute	Data type	Description
-----------	-----------	-------------

Id	int (PK)	Primary key, automatically generated
BookingNumber	string(20)	Unique, human-readable reference generated after insertion, for example BK-000042
CustomerId	int (FK)	Foreign key referencing the requesting Customer
PropertyId	int (FK)	Foreign key referencing the requested Property
Type	enum	Purchase or Rental
PreferredPaymentMethod	enum	Cash, BankTransfer, MobileMoney, or Card
ProposedAmount / ProposedDeposit	decimal(18,2), nullable	Amount and deposit proposed by the customer
Status	enum	Pending, Confirmed, or Rejected
RequestDate	DateTime	Date the request was submitted
ReviewedById / ReviewedDate / ReviewNotes	string (FK) / DateTime / string, nullable	Staff member who reviewed the request, when this occurred, and any remarks
ResultingSaleId / ResultingRentalId	int (FK), nullable	Links to the Sale or Rental created if the request was confirmed

All monetary attributes were declared using the decimal(18,2) column type rather than a floating-point type, in order to avoid the rounding errors that binary floating-point representations can introduce into financial calculations. Foreign key relationships, for example between Payment and Sale, or between Property and Project, were declared using Entity Framework Core's fluent configuration in ApplicationDbContext, ensuring that the database itself rejects orphaned records, in line with the referential integrity principle discussed in Section 2.1.

4.6 Implementation of modules

The system is organised into fourteen functional modules, each represented by a dedicated MVC controller and, where the module involves non-trivial business rules, a corresponding service class. Table 4.8 lists the five default user-role permissions implemented through ASP.NET Core Identity and the modules each role is authorised to access.

Table 4.8: System user roles and permissions

Role	Primary permissions
Administrator	Full access to every module, including user and role management, and system-wide reports
Project Manager	Create and update projects and project tasks, manage properties, view reports relevant to project progress
Sales Agent	Register customers, record sales and rentals, upload related documents
Accountant	Record and review payments, generate financial reports

Customer	View the sale or rental agreement, and related payment history, linked to the customer's own account
----------	--

The Projects module allows Administrators and Project Managers to create a project with a name, code, location, budget, and schedule, and to monitor its status, which ranges from Planning through InProgress to Completed, OnHold, or Cancelled, along with a numeric progress percentage. The Project Tasks module allows individual tasks within a project to be assigned to a specific employee, given a priority level and a due date, and tracked through Pending, InProgress, Completed, or Delayed states. Any change made to a project's status or progress percentage is additionally recorded as a ProjectStatusHistory entry, capturing the previous and new values, the reason given by the user making the change, and the identity of that user, providing an auditable history of how each project has progressed.

The Properties module allows staff to register individual units, of type Apartment, Villa, Office, Shop, or Land, under a specific project, upload one or more images for each unit, and track each unit's status as it moves from Available or UnderConstruction through to Sold or Rented. The Customers module maintains a directory of buyers and tenants, optionally linked to a self-service login account, which the Customer role uses to view personal transaction history.

The Sales and Rentals modules are implemented through the SalesService and RentalService classes respectively, both of which enforce the rule that a property already marked Sold or Rented cannot be sold or rented again. As illustrated in the code excerpt reproduced in Appendix A, creating a sale automatically changes the associated property's status to Sold and, where a deposit was specified, immediately creates a corresponding Payment record; deleting a sale reverses this by releasing

the property back to Available status and removing any payments linked to that sale.

The Payments module, implemented through PaymentService, is deliberately kept independent of the SalesService and RentalService classes with respect to balance calculation: whenever a payment is created or deleted, PaymentService calls a dedicated recalculation method on the relevant sales or rental service, which sums all payments linked to that sale or rental and updates the remaining balance and payment status accordingly, rather than trusting the calling code to keep these values consistent. This design decision, referenced earlier in Section 3.4, ensures that the payment status of every sale and rental is always derived directly from its actual payment history.

The Documents module allows a file, such as a signed sales contract or a payment receipt, to be uploaded and optionally linked to a project, property, sale, rental, or customer record, using the FileStorageService to manage physical storage under wwwroot/uploads. The Notifications module runs a background check, executed on every dashboard load through INotificationService.GenerateSystemNotificationsAsync, which scans for pending payments, contracts expiring within thirty days, and projects that have fallen behind schedule, and raises an in-application notification, shown to the user through the notification-bell view component, for each condition detected. Each user may control, through a dedicated notification-settings page, which categories of notification they wish to receive, an opt-out model in which a notification type is treated as enabled unless the user has explicitly disabled it. The Reports module, implemented through IExportService, aggregates data across the other modules to produce sales, revenue, property, customer, payment, project-progress, and rental reports, each exportable to PDF using QuestPDF or to Excel using ClosedXML. Finally, the Users module, restricted to the Administrator role, allows new staff

accounts to be created and assigned to one of the five roles defined in `Models/Roles.cs`.

The Booking Requests module implements a customer self-service workflow additional to the modules described above. Any registered Customer may browse a public catalogue of available properties, implemented through `PublicController`, and submit a booking request specifying a preferred payment method and a proposed amount and deposit. `BookingRequestService` validates that the selected property is still Available and that the customer does not already have a pending request for it, and, once a Project Manager or Sales Agent reviews the request, either confirms it, converting it into a Sale or a Rental through the corresponding service and recording the resulting record's identifier against the booking request, or rejects it with a review note visible to the customer.

The Role Permissions module, restricted to the Administrator role, allows the default module-to-role access scheme summarised in Table 4.8 to be adjusted at run time. `PermissionService` builds a permission matrix from the `RolePermission` table, caches it in memory for five minutes to limit repeated database access, and is consulted by `ModulePermissionAttribute` on every request to a permission-controlled module; the Administrator's access is hard-coded as complete and is not itself configurable through this mechanism.

CHAPTER FIVE: SYSTEM IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter presents the outcome of implementing the design described in Chapter Four, and documents the testing activities carried out to verify that the resulting system behaves correctly. The system was implemented incrementally, module by module, following the iterative cycle described in Section 3.5, and each module was exercised manually using the demonstration accounts and seed data generated automatically by the application's DbInitializer component on first run. Screenshots of the running system, captured directly from the deployed application, are presented in Section 5.3 to illustrate the completed interface.

5.2 Testing

5.2.1 Unit-level verification

Because the correctness of the system depends heavily on a small number of business rules rather than on complex algorithms, testing focused on directly verifying those rules through the service layer described in Section 4.6. Table 5.1 summarises representative test cases exercised against the SalesService, RentalService, and PaymentService classes, each of which was confirmed to behave as expected.

Table 5.1: Sample unit test cases and results

Test ID	Scenario	Expected result	Actual result
---------	----------	-----------------	---------------

T-01	Create a sale for an available property	Property status changes to Sold; deposit recorded as an initial payment	Passed
T-02	Attempt to sell a property already marked Sold	Operation rejected with an explanatory error message	Passed
T-03	Delete an existing sale	Property status reverts to Available; associated payments removed	Passed
T-04	Record a partial payment against a sale	Remaining balance decreases; payment status changes to Partial	Passed
T-05	Record payments summing to the full final price	Payment status changes to Paid; remaining balance equals zero	Passed
T-06	Delete a payment previously recorded against a sale	Remaining balance and payment status recalculated to reflect the deletion	Passed
T-07	Create a rental for a property already marked Rented	Operation rejected with an explanatory error message	Passed
T-08	Dashboard load with a contract expiring within thirty days	An expiring-contract notification is generated	Passed

5.2.2 User acceptance testing

User acceptance testing was carried out by exercising the complete system end to end using each of the six demonstration accounts described in the project's README file, one for each of the five roles, covering login, navigation, and the principal task associated with that role. Table 5.2 summarises the outcome of this testing.

Table 5.2: Sample user acceptance testing results

UAT ID	Role tested	Task performed	Outcome
UAT-01	Administrator	Log in, view dashboard, create a new staff user	Completed successfully
UAT-02	Project Manager	Create a project and assign a task to an employee	Completed successfully
UAT-03	Sales Agent	Register a new customer and record a property sale	Completed successfully
UAT-04	Accountant	Record a payment against an existing sale and review the payment report	Completed successfully
UAT-05	Customer	Log in and view own sale record and payment history	Completed successfully

The testing activities above confirmed that the core business rules identified in Chapter Four were correctly enforced across every implemented module, and that each of the five user roles could complete its intended tasks without encountering unexpected errors.

5.3 Screenshots and explanation

This section presents annotated screenshots of the completed system, captured directly from the running application using the Administrator account, in order to demonstrate that the functionality described in Chapter Four was successfully implemented.

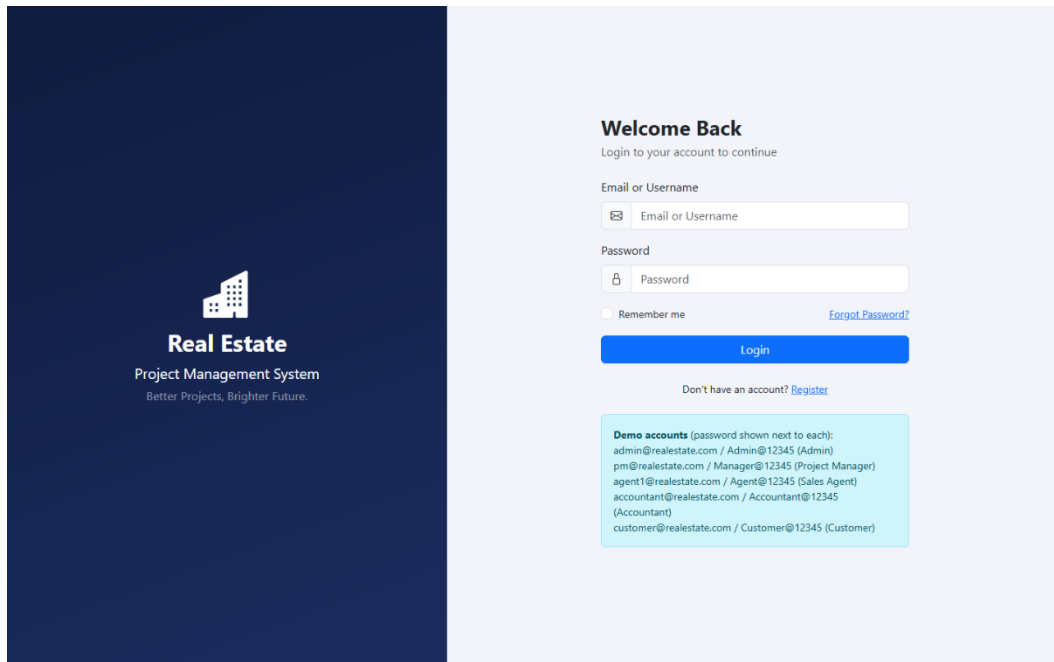


Figure 5.1: Login screen, showing the split-panel layout and the demonstration account credentials made available for evaluation purposes

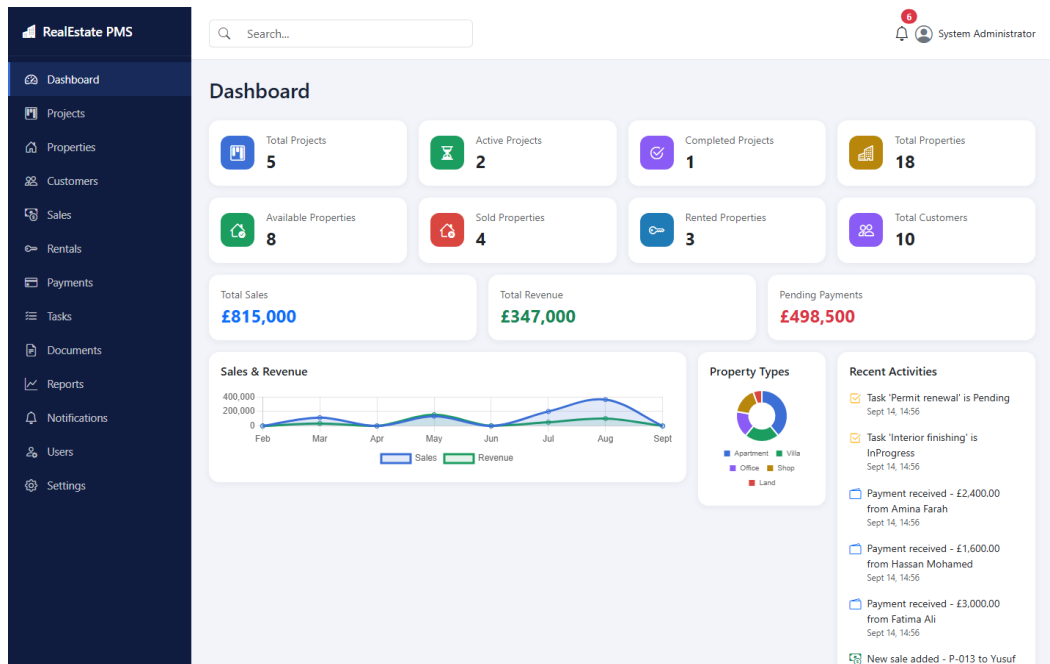


Figure 5.2: Administrator dashboard, summarising project, property, sales, and revenue statistics, together with a sales-and-revenue trend chart, a property-type distribution chart, and a recent-activity feed

RealEstate PMS

Dashboard

Projects

Properties

Customers

Sales

Rentals

Payments

Tasks

Documents

Reports

Notifications

Users

Settings

Search...

6

System Administrator

Properties

+ Add Property

Search properties...

All Types

All Statuses

All Projects

Filter

Image	Code	Project	Type	Price	Status	Actions
	P-015	Beach Front Villas	Villa	£480,000	UnderConstruction	
	P-014	Beach Front Villas	Villa	£500,000	Available	
	P-013	City Center Complex	Villa	£380,000	Sold	
	P-012	City Center Complex	Apartment	£130,000	Rented	
	P-011	City Center Complex	Apartment	£140,000	Available	
	P-017	Somali Plaza	Shop	£90,000	Rented	
	P-010	Somali Plaza	Office	£210,000	Sold	
	P-005	Somali Plaza	Land	£450,000	Available	

Previous

1

2

3

Next

Figure 5.3: Properties list, showing property code, associated project, type, price, and current status, with search and filter controls

RealEstate PMS

Dashboard

Projects

Properties

Customers

Sales

Rentals

Payments

Tasks

Documents

Reports

Notifications

Users

Settings

Search...

6

System Administrator

Add Property

Property Code

Project

Property Type

-- Select Project --

Apartment

Building

Floor

Unit Number

Area (sqm)

0.00

Bedrooms

Bathrooms

Price

0

0

0.00

Status

Choose Images

Available

Choose Files

No file chosen

Description

Save Property

Cancel

Figure 5.4: Add new property form, used to register a property under a selected project

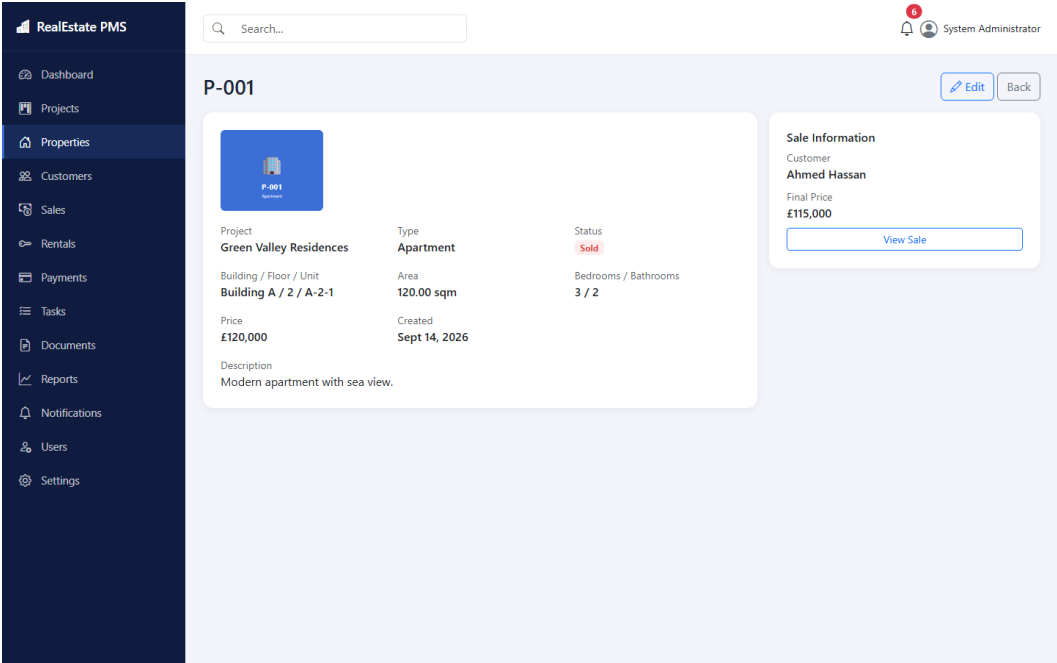


Figure 5.5: Property details view, showing full unit information alongside the linked sale record for a sold property

RealEstate PMS

Dashboard
Projects
Properties
Customers
Sales
Rentals
Payments
Tasks
Documents
Reports
Notifications
Users
Settings

Search...

System Administrator

Customers

+ Add Customer

Search customers...

All Countries

Filter

Name	Phone	Email	Nationality	Actions
Khadija Omar	+252614445566	khadija.omar@example.com	Somali	
Sagal Warsame	+252612223344	sagal.warsame@example.com	Somali	
Abdullahi Nur	+252619990011	abdullahi.nur@example.com	Somali	
Amina Farah	+252617778899	amina.farah@example.com	Somali	
Hassan Mohamed	+252615556677	hassan.mohamed@example.com	Somali	
Yusuf Ali	+252613332222	yusuf.ali@example.com	Somali	
Maryam Ahmed	+252614448888	maryam.ahmed@example.com	Somali	
Mohamed Yusuf	+252611888888	mohamed.yusuf@example.com	Somali	

Previous

1

2

Next

Figure 5.6: Customers list, showing registered buyers and tenants

RealEstate PMS

Dashboard
Projects
Properties
Customers
Sales
Rentals
Payments
Tasks
Documents
Reports
Notifications
Users
Settings

Search...

System Administrator

Sales Management

+ Add Sale

Search by customer or property...

All Statuses

Filter

Customer	Property	Sales Agent	Sale Date	Final Price	Status	Actions
Yusuf Ali	P-013	Said Ibrahim	2026-08-14	£365,000	Partial	
Mohamed Yusuf	P-010	Said Ibrahim	2026-07-14	£200,000	Partial	
Maryam Ahmed	P-006	Mohamed Ali	2026-05-14	£135,000	Paid	
Ahmed Hassan	P-001	Mohamed Ali	2026-03-14	£115,000	Partial	

Figure 5.7: Sales management, listing recorded sales with customer, property, agent, sale date, final price, and payment status

RealEstate PMS

Search...

System Administrator

Rental Management

+ Add Rental

Search by customer or property... All Contract Statuses Filter

Tenant	Property	Monthly Rent	Start	End	Payment	Contract	Actions
Amina Farah	P-012	£1,200	2026-08-14	2027-08-14	Partial	Active	View Edit Delete
Hassan Mohamed	P-017	£800	2026-07-14	2027-07-14	Partial	Active	View Edit Delete
Fatima Ali	P-003	£1,500	2026-06-14	2027-06-14	Partial	Active	View Edit Delete

Figure 5.8: Rentals management, listing active rental agreements

RealEstate PMS

Dashboard

Projects

Properties

Customers

Sales

Rentals

Payments

Tasks

Documents

Reports

Notifications

Users

Settings

Search...

System Administrator

Payments

+ Add Payment

Search by customer...

All Types

Filter

Customer	Type	Amount	Date	Method	Status	Actions
Amina Farah	Rental #3	£2,400	2026-08-14	Cash	Completed	View Delete
Yusuf Ali	Sale #4	£100,000	2026-08-14	Cash	Completed	View Delete
Hassan Mohamed	Rental #2	£1,600	2026-07-14	Cash	Completed	View Delete
Mohamed Yusuf	Sale #3	£50,000	2026-07-14	BankTransfer	Completed	View Delete
Fatima Ali	Rental #1	£3,000	2026-06-14	Cash	Completed	View Delete
Maryam Ahmed	Sale #2	£135,000	2026-05-14	Cash	Completed	View Delete
Ahmed Hassan	Sale #1	£20,000	2026-05-14	MobileMoney	Completed	View Delete
Ahmed Hassan	Sale #1	£35,000	2026-03-14	BankTransfer	Completed	View Delete

Figure 5.9: Payments list, showing individual payment transactions recorded against sales and rentals

RealEstate PMS

Dashboard

Projects

Properties

Customers

Sales

Rentals

Payments

Tasks

Documents

Reports

Notifications

Users

Settings

Search...

System Administrator

Projects

+ Add Project

Search projects...

All Statuses

Filter

#	Project Name	Location	Status	Progress	Actions
PRJ-005	Beach Front Villas	Kismayo	OnHold	20%	View Edit Delete
PRJ-004	City Center Complex	Mogadishu	InProgress	45%	View Edit Delete
PRJ-003	Somali Plaza	Bosaso	Completed	100%	View Edit Delete
PRJ-002	Horizon Towers	Hargeisa	Planning	15%	View Edit Delete
PRJ-001	Green Valley Residences	Mogadishu	InProgress	70%	View Edit Delete

Figure 5.10: Projects list, showing project code, location, status, and progress

RealEstate PMS

Search...

System Administrator

Project Tasks

+ Add Task

Search tasks... All Statuses All Projects Filter

Task	Project	Assigned To	Due Date	Priority	Status	Progress	Actions
Permit renewal	Beach Front Villas	Amina Yusuf	2026-09-24	Urgent	Pending		Edit Delete
Interior finishing	City Center Complex	Amina Yusuf	2026-11-14	Medium	InProgress		Edit Delete
Plumbing rough-in	City Center Complex	Amina Yusuf	2026-08-30	High	Delayed		Edit Delete
Final inspection	Somali Plaza	Amina Yusuf	2026-08-14	Medium	Completed		Edit Delete
Structural approval	Horizon Towers	Amina Yusuf	2026-10-04	High	Pending		Edit Delete
Site survey	Horizon Towers	Amina Yusuf	2026-09-09	Urgent	Delayed		Edit Delete
Electrical installation	Green Valley Residences	Amina Yusuf	2026-10-14	Medium	InProgress		Edit Delete
Foundation works	Green Valley Residences	Amina Yusuf	2026-03-14	High	Completed		Edit Delete

Figure 5.11: Project tasks, showing individual tasks assigned to employees within a project

RealEstate PMS

Dashboard
Projects
Properties
Customers
Sales
Rentals
Payments
Tasks
Documents
Reports
Notifications
Users
Settings

Search...

System Administrator

Documents

Search documents...

All Types

Filter

Title	Type	Related To	Uploaded By	Date	Actions
Payment Receipt - Payment #1	PaymentReceipt		System Administrator	2026-09-14	
Project Document - Green Valley Residences	ProjectDocument	Project: Green Valley Residences	System Administrator	2026-09-14	
Property Document - P-001	PropertyDocument	Property: P-001	System Administrator	2026-09-14	
Rental Contract - Rental #1	RentalContract	Rental #1	System Administrator	2026-09-14	
Sales Contract - Sale #1	SalesContract	Sale #1	System Administrator	2026-09-14	

Upload Document

Figure 5.12: Documents list, showing uploaded contracts and receipts linked to their related records

RealEstate PMS

Dashboard
Projects
Properties
Customers
Sales
Rentals
Payments
Tasks
Documents
Reports
Reports
Notifications
Users
Settings

Search...

System Administrator

Reports

Sales Report

Revenue Report

Property Report

Customer Report

Payment Report

Project Progress Report

Rental Report

Ixii

Figure 5.13: Reports hub, providing access to the sales, revenue, property, customer, payment, project-progress, and rental reports described in Section 4.6

Name	Email	Roles	Status	Actions
Ahmed Hassan	customer@realestate.com	Customer	Active	Edit Delete
Amina Yusuf	pm@realestate.com	ProjectManager	Active	Edit Delete
Hodan Warsame	accountant@realestate.com	Accountant	Active	Edit Delete
Mohamed Ali	agent1@realestate.com	SalesAgent	Active	Edit Delete
Said Ibrahim	agent2@realestate.com	SalesAgent	Active	Edit Delete
System Administrator	admin@realestate.com	Admin	Active	Edit

Figure 5.14: Users management, available to the Administrator role for creating and reviewing staff accounts

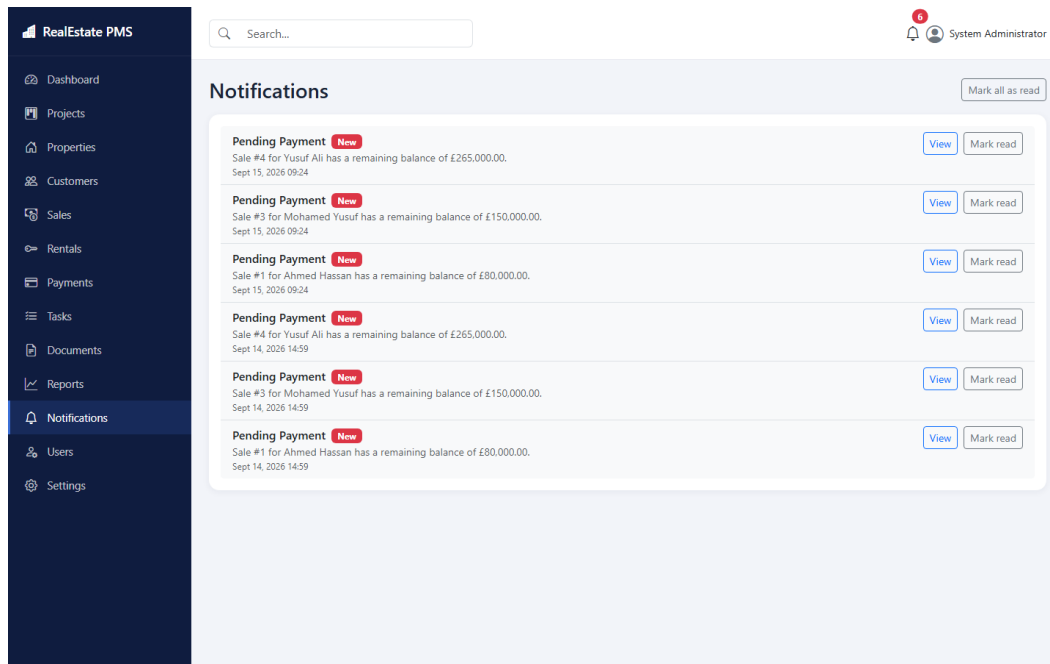


Figure 5.15: Notifications panel, displaying system-generated alerts for pending payments and other conditions described in Section 4.6

Taken together, the screenshots above confirm that every module identified in Section 4.6 was implemented and is fully operational, and that the role-based access control and automatic business rules described throughout Chapter Four function as designed in the completed application.

CHAPTER SIX: CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

This study set out to design, develop, and evaluate a web-based Real Estate Project Management System capable of replacing the manual and spreadsheet-based methods that many small real estate companies currently depend on. The completed system, built using ASP.NET Core 9, Entity Framework Core 9, and Microsoft SQL Server, integrates project tracking, property inventory, customer records, sales and rental transactions, payment processing, document storage, reporting, and system-generated notifications into a single, role-aware web application. Testing confirmed that the core business rules identified during requirements analysis, including automatic property status updates and automatic payment balance recalculation, function correctly, and that each of the five defined user roles can complete its intended tasks. The study therefore concludes that its general and specific objectives, stated in Section 1.3, were achieved.

6.2 Summary of findings

The requirements analysis confirmed that the central weaknesses of manual and spreadsheet-based real estate record keeping are outdated property status information, error-prone manual balance calculation, the absence of automatic warnings for overdue payments or expiring contracts, and scattered document storage, as discussed in Section 1.2. The system design and implementation described in Chapters Three and Four directly addressed each of these weaknesses: property status is now updated automatically whenever a sale or rental is created or removed, payment balances are recalculated automatically after every payment transaction, notifications are generated automatically for pending payments, expiring contracts, and delayed projects, and documents are stored centrally and

linked to their related records. The testing results presented in Chapter Five confirmed that these mechanisms operate correctly across the demonstration data used for evaluation.

6.3 Recommendations and suggestions

Based on the findings summarised above, it is recommended that any real estate company considering adoption of a system of this kind first document its own specific business rules and approval workflows before configuring the software, since even a well-designed generic system cannot anticipate every local business practice. It is further recommended that staff receive brief, structured training on the system before its introduction, particularly regarding the payment recording workflow, since the accuracy of the automatically calculated balances depends entirely on every payment being entered promptly and completely. Finally, it is recommended that a company adopting this system perform regular database backups, since, unlike a collection of paper files, the entire operational record of the company would reside in a single database.

6.4 Future enhancement

Several directions for future enhancement were identified but fall outside the scope defined in Section 1.4. A dedicated mobile application, or a responsive customer-facing portal, would allow customers to review their payment status without needing to contact staff directly. Integration with short message service and electronic mail providers would allow the existing in-application notifications described in Section 4.6 to also reach staff and customers outside the application itself. Integration with mobile money and banking application programming interfaces would allow payments to be recorded automatically rather than manually. Finally, extending the system to support multiple branches or multiple

independent companies within a single deployment, using a tenant identifier added to the core entities described in Section 4.5, would broaden the system's applicability beyond the single-organisation scope adopted in this study.

REFERENCES

- Baum, A. (2020). *PropTech 2020: The future of real estate*. University of Oxford, Saïd Business School.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley.
- Freeman, A. (2023). *Pro ASP.NET Core 8* (10th ed.). Apress.
- International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering*. ISO.
- Microsoft. (2025). *ASP.NET Core documentation*.
<https://learn.microsoft.com/aspnet/core>
- Microsoft. (2025). *Entity Framework Core documentation*.
<https://learn.microsoft.com/ef/core>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Stellman, A., & Greene, J. (2014). *Learning agile: Understanding Scrum, XP, Lean, and Kanban*. O'Reilly Media.

APPENDICES

Appendix A: Sample source code

The excerpt below, reproduced from `Services/SalesService.cs`, implements the business rule described in Section 4.6, under which creating a sale automatically updates the property's status and records any deposit as an initial payment.

```
public async Task<(bool Success, string? Error)> CreateSaleAsync(Sale sale)
{
    var property = await _context.Properties.FindAsync(sale.PropertyId);

    if (property == null)
        return (false, "Property not found.");

    if (property.Status is PropertyStatus.Sold or PropertyStatus.Rented)
        return (false, "This property is not available for sale.");

    sale.FinalPrice = sale.TotalPrice - sale.Discount;

    sale.RemainingBalance = sale.FinalPrice - sale.Deposit;

    sale.PaymentStatus = CalculateStatus(sale.FinalPrice, sale.Deposit);

    property.Status = PropertyStatus.Sold;
```

```
        _context.Sales.Add(sale);

        await _context.SaveChangesAsync();

        if (sale.Deposit > 0)
        {
            _context.Payments.Add(new Payment
            {
                CustomerId = sale.CustomerId,

                SaleId = sale.Id,

                Amount = sale.Deposit,

                PaymentDate = sale.SaleDate,

                Method = PaymentMethod.Cash,

                Notes = "Initial deposit recorded automatically with the sale."
            });

            await _context.SaveChangesAsync();
        }

        return (true, null);
    }
}
```

The excerpt below, reproduced from `Services/PaymentService.cs`, shows how recording or deleting a payment triggers automatic recalculation of the associated sale or rental balance, as described in Section 4.6.

```
public async Task<(bool Success, string? Error)> CreatePaymentAsync(Payment
payment)
{
    if (payment.SaleId == null && payment.RentalId == null)
        return (false, "A payment must be linked to a Sale or a Rental.");

    _context.Payments.Add(payment);

    await _context.SaveChangesAsync();

    if (payment.SaleId.HasValue)
        await
        _salesService.RecalculatePaymentStatusAsync(payment.SaleId.Value);

    if (payment.RentalId.HasValue)
        await
        _rentalService.RecalculatePaymentStatusAsync(payment.RentalId.Value);

    return (true, null);
}
```

Appendix B: Demonstration accounts used for testing

Role	E-mail	Password
Administrator	admin@realestate.com	Admin@12345
Project Manager	pm@realestate.com	Manager@12345
Sales Agent	agent1@realestate.com	Agent@12345
Accountant	accountant@realestate.com	Accountant@12345
Customer	customer@realestate.com	Customer@12345

Appendix C: Project folder structure

RealEstatePMS/

 Controllers/ MVC controllers (one per module)

 Models/

 Entities/ EF Core entities

 Enums/ Status / type enums

 Data/ ApplicationDbContext + DbInitializer (seed data)

 Repositories/ Generic repository (IRepository<T> / Repository<T>)

 Services/ Business logic (Sales, Rentals, Payments,
Notifications, Export, File storage, Dashboard)

ViewModels/	View-specific models
ViewComponents/	NotificationBell dropdown
Helpers/	PaginatedList<T>, StatusBadge
Views/	Razor views, organised by controller
wwwroot/	Static assets; uploaded files under wwwroot/uploads
Migrations/	EF Core Code-First migrations

Appendix D: System screenshots

Additional system screenshots are included in Section 5.3 of this thesis.

lxxiii

lxxiii